

# Understanding JEV: Core Decision-Making Primitives in RAG Pipelines

## 1. Introduction: The Need for Decision Points in RAG

Modern Retrieval-Augmented Generation (RAG) pipelines are designed to retrieve relevant information from a knowledge base and feed it to a Language Model (LLM) to generate accurate answers. Typically, initial retrieval relies on vector search or full-text search algorithms. However, standard vector searches measure semantic proximity - how closely words or concepts resemble each other in a vector space - rather than correctness, compliance, or alignment with business logic. "Vector distances can only measure semantic proximity, but not correctness under your business logic." When organizational policies change, official documentation updates, or distinct operational criteria are required (such as prioritizing authoritative architectural guides over quick workarounds), pure semantic search falls short. A robust RAG pipeline requires explicit decision points where retrieved context can be evaluated, filtered, and prioritized according to active business rules. Key decision points across RAG workflows include:

**Reranking:** Re-ordering retrieved document chunks based on active, context-aware rules.

**Filtering:** Discarding context that falls below strict quality, relevance, or factual thresholds.

**Citations:** Verifying whether a specific passage directly validates a generated response.

**Entity Validation:** Confirming the validity of extracted entities within structured knowledge representations like GraphRAG. To handle these critical evaluation steps effectively, JEV introduces three fundamental building blocks designed to make explicit, rule-based choices within AI workflows.

## 2. The Three Core Primitives of JEV

JEV provides three core decision-making primitives that developers can assemble to introduce explicit logic into retrieval pipelines. Each primitive addresses a distinct decision pattern:

**Binary Decision Making (True/False Classifier):** This primitive uses JEV's null criteria mechanism to calculate calibrated confidence probabilities across binary outcomes (True vs. False). Rather than returning a generic boolean, null acts as the underlying primitive mechanism that assigns calibrated probabilities to evaluate whether an assertion is met (e.g., True = the passage directly answers the question; False = the passage shares words or topics but does not answer it). This allows developers to set precise acceptance or discard thresholds based on confidence scores.

**Choice Ranker:** This primitive evaluates multiple discrete choices against defined criteria (acting as executable policies) to select a single optimal option out of competing alternatives.

Score Ladder: This primitive evaluates a list of options and assigns relative score levels across them to establish a structured, ordered priority sequence. Understanding when to select a Choice Ranker versus a Score Ladder comes down to the structural output required by the workflow: Choice Ranker is selected when a system must isolate a single optimal path out of discrete alternatives (e.g., selecting the single valid policy version among competing docs), whereas a Score Ladder is used when re-ordering an entire collection of contexts (e.g., preparing a ranked top-K chunk context window for an LLM prompt).

| Primitive Name         | Core Mechanism                                                                                   | Primary RAG Use Case                                                                                                                   |
|------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Binary Decision Making | Calculates calibrated probabilities for True/False conditions using the null criteria mechanism. | Filtering & Quality Control: Setting strict probability thresholds to retain direct answers and discard keyword-overlapping noise.     |
| Choice Ranker          | Evaluates a set of discrete choices against executable policies to select a single best option.  | Dynamic Selection: Selecting a single optimal policy version or document format (e.g., official v2 docs vs. legacy guides).            |
| Score Ladder           | Assigns relative score levels across multiple options to construct an ordered priority list.     | Context Prioritization: Re-ordering an entire list of retrieved chunks prior to passing them into the final LLM prompt context window. |

With these three theoretical primitives established, we can now examine how they operate in real-world scenarios to enable steerable reranking through executable policies.

### 3. Deep Dive: Steerable Reranking and Executable Policies

In conventional pipelines, instructing a model to re-evaluate documents involves constructing long, complex natural language prompts that can be unreliable. JEV shifts this paradigm by converting evaluation criteria into executable policies that establish strict decision boundaries. To see this in action, consider a practical documentation scenario where a developer queries: "How do I fix a 401 unauthorized error from the APIs?" In a standard knowledge base, initial vector retrieval might return multiple chunks containing identical keywords (401, unauthorized, API), including older workarounds and newer official standards. By applying JEV, developers can steer how these retrieved chunks are evaluated and ranked without modifying the underlying text chunks or altering pipeline code logic:

**Policy-Driven Output Steerability:** If the pipeline enforces an "Authoritative Version 2" policy, JEV evaluates candidate chunks against criteria requiring official, up-to-date protocol fixes. Under this policy, JEV ranks a strict, official documentation chunk as the top result. Conversely, if the system switches to a "Fast Workaround" policy, JEV shifts the output ranking to prioritize a quick copy-paste code snippet chunk for that same 401 unauthorized query.

**Strict Decision Boundaries over Loose Prompts:** Instead of relying on vague prompt instructions, JEV criteria establish explicit evaluation conditions. Using the null primitive, JEV assigns a high probability to passages providing direct solutions (e.g., "API keys must be passed in the Authorization header via Bearer tokens" ) while penalizing passages that merely mention authorization errors without offering a solution.

Calibrated Confidence Filtering: Because JEV outputs calibrated confidence scores (e.g., distinguishing direct answer statements like "refunds are processed within 14 days" from general policy mentions), systems can automatically discard low-confidence context chunks before they reach the generation phase. Understanding how executable policies steer results raises a fundamental architectural question: how does JEV compare to traditional reranking methods like Cross-Encoders or LLM-based rerankers?

#### 4. Architectural Comparison: JEV vs. Cross-Encoders vs. LLMs

Choosing the right decision component requires balancing steerability, runtime latency, and scaling costs. Below is an architectural comparison evaluating JEV against traditional Cross-Encoders and LLM-as-a-Reranker models (e.g., Gemini Flash / Flashlite):

| Reranking Approach                     | Instruction Following (Steerability)                                               | Cost & Speed Efficiency                                                                                          | Key Limitation / Advantage                                                                                                        |
|----------------------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Cross-Encoders                         | None: Evaluates document-query pairs directly; cannot follow dynamic instructions. | High Speed / Low Cost: Fast processing, but static evaluation.                                                   | Limitation: "Frozen in time"-cannot adapt when business policies change without full model retraining.                            |
| LLM-as-a-Reranker (e.g., Gemini Flash) | High: Fully capable of following complex criteria and instruction sets.            | Low Efficiency: Prohibitively expensive and slow; costs scale linearly as text chunk sizes expand.               | Limitation: High cost and latency bottlenecks make scaling across large text chunks impractical for high-throughput applications. |
| JEV                                    | High: Highly steerable via executable criteria and dynamic policy definitions.     | High Efficiency: Low cost and low latency; runtime speed and cost remain flat/stable even as chunk sizes expand. | Advantage: Combines instruction-following steerability with extreme cost stability, high concurrency, and robust accuracy gains.  |

#### Key Performance & Benchmark Metrics

**High-Concurrency Performance:** Running decision evaluations sequentially (e.g., at 17 decisions per second) requires approximately 40 seconds to process a query set. By leveraging JEV's 64 parallel pools, processing time drops to 7.6 seconds, demonstrating high concurrency efficiency.

**Chunk Size Scaling Behavior:** While LLM rerankers (such as Gemini Flashlite) suffer from linear cost and latency increases as context chunk sizes expand, JEV maintains flat, stable performance and low costs even when processing significantly larger text chunks.

**Retrieval Accuracy Improvements:** On full-text search baselines (such as BM25 full-text search yielding a baseline 21% top-1 accuracy), adding JEV as an instruction-following reranker elevates top-1 retrieval accuracy to 54%. Note that top-1 accuracy is a strict metric; in typical top-K retrieval evaluations (e.g., measuring context presence within the top 10 or 20 retrieved chunks), JEV achieves even higher accuracy gains. Having evaluated JEV's performance and architectural strengths, let us summarize the key takeaways for applying JEV within production AI workflows.

#### 5. Summary & Key Takeaways for Learners

**Implement Explicit Decision Engines at Critical Pipeline Junctions:** Vector search only measures semantic proximity, not correctness or policy adherence. Deploy dedicated decision engines like JEV at critical RAG pipeline junctions such as reranking, filtering, citation verification, and entity validation to enforce business logic.

Leverage Executable Policies for Steerable Reranking: Formulate criteria as explicit, executable policies (e.g., toggling between an "Authoritative Version 2" policy and a "Fast Workaround" policy). This allows you to dynamically steer document ranking for queries like API error fixes without modifying underlying context data or application code.

Optimize Cost, Latency, and Accuracy at Scale: While LLM rerankers offer steerability, their costs and latency scale linearly with chunk size expansion. JEV delivers high steerability at extremely low costs with stable scaling across large chunks, utilizing parallel concurrency (64 pools) to cut execution time from 40s to 7.6s while boosting top-1 retrieval accuracy from 21% to 54%.